

Architecture Decision Record

ADR-031 - Event Streaming Platform Selection

Status	Accepted
Date	September 2025
Author	Jennifer Watson
Reviewers	Raj Krishnamurthy, Sandra Chen, Marcus Webb
Project	Phoenix - monolith to microservices migration

Context

Project Phoenix decomposes a Java and Spring Boot monolith into 14 services. Inter-service communication must be asynchronous and event-driven to allow independent deployment. We evaluated three streaming platforms against operational maturity, team experience, ecosystem, and total cost.

Options

Criterion	Kafka	Pulsar	NATS
Team experience	High	Low	Low
Operational maturity	High	Medium	Medium
Ecosystem and tooling	Broad	Growing	Lean
Ordering guarantees	Strong	Strong	Configurable
Operational complexity	Medium	High	Low

Decision

Adopt Apache Kafka as the event streaming layer for all Phoenix services. The deciding factor is operational depth, the team already runs Kafka in production and can debug it under pressure. NATS was attractive for its simplicity but lacked the replay and retention guarantees the payment and reconciliation domains require. Pulsar was technically strong but added operational surface the team would learn during a migration, which is the wrong time.

Weighted Scoring

Each criterion was scored one to five and weighted by its importance to a migration under a reliability target. Kafka led on the two heaviest weights, team experience and operational maturity.

Criterion	Weight	Kafka	Pulsar	NATS
Team experience	0.30	5	2	2
Operational maturity	0.25	5	3	3
Ecosystem and tooling	0.20	5	3	2
Ordering and replay	0.15	5	5	3
Operational simplicity	0.10	3	2	5
Weighted total	1.00	4.80	2.85	2.65

Cost Considerations

All three run on the existing EKS footprint, so the dominant cost is operational time rather than license or hardware. Kafka carries a broker and storage cost already absorbed in the current AWS spend. The event layer growth is expected to sit inside the projected 15 percent migration increase.

Risks and Mitigations

Risk	Mitigation
Broker misconfiguration under load	Standard topic templates and reviews
Consumer lag during cutover	Backpressure and lag dashboards
Schema drift across services	Registry enforcement on every topic
Replay of poison messages	Dead-letter topics and idempotency keys

Consequences

Standardize on per-service topics with schema registry enforcement. Document replay and idempotency expectations for all consumers. This decision supersedes informal Pulsar exploration from the discovery phase.

Implementation Notes

- One logical topic per service domain, partitioned by aggregate key.
- Consumers must be idempotent and tolerate at-least-once delivery.
- A dead-letter topic accompanies every consumer group.
- Retention is set per domain, longer for payment and reconciliation.

Related Records

ADR	Title	Relationship
ADR-031	Event streaming platform selection	This record
ADR-048	Decoupling the payment processing service	Depends on this
ADR-022	Service ownership and topic naming	Referenced

Maintained in Confluence alongside the platform decision log.